AI-Powered Fraud Detection System for Online Transactions

Complete Comprehensive Guide: Local Execution, GitHub Publishing & Vercel Cloud Deployment

1. Executive Project Overview

This project is an advanced, production-grade **Dual-Path AI Fraud Detection System** designed specifically for modern digital payment systems (UPI, Credit/Debit Cards, Netbanking). It incorporates an **Ensemble Risk Scoring Engine** combining tree-based supervised machine learning (XGBoost / Random Forest), recurrent neural anomaly detection (LSTM / Autoencoder), and heuristic compliance overlays mapped to **OWASP API Security Top 10 (2023)** and **PCI DSS v4.0** requirements.

Key Capability	Implementation Architecture
Supervised Scoring (45%)	XGBoost / Random Forest with SMOTE balancing tabular features
Deep Sequence Scoring (40%)	LSTM & Autoencoder sequence window ($T=10$) for temporal ATO detection
Rule Overlay (15%)	Velocity bursts, device mismatch penalties, high-value heuristics
Decision Policy	Calibrated 3-tier action: **Approve** (<35), **Step-Up 2FA** (35-70), **Block** (>70)
Currency Ecosystem	Calibrated for Indian Rupee (INR ■) UPI, Food Apps, E-Commerce, Gaming

2. System Prerequisites & Environment

Before starting, ensure the recipient's computer has the following tools installed:

- **Python 3.10, 3.11, or 3.12** installed (check with python --version). Ensure *Add Python to PATH* was checked during installation.
- **Git** installed (check with git --version) for version control and GitHub operations.
- **Modern Web Browser** (Google Chrome, Microsoft Edge, Mozilla Firefox, or Brave).
- **Code Editor / Terminal** (VS Code, Windows PowerShell, Terminal on macOS/Linux).

3. Step-by-Step Guide: Running the Project Locally

Follow these exact steps to run the complete fraud detection web dashboard and real-time transaction simulator on any Windows, macOS, or Linux machine.

Step 3.1 — Extract or Navigate to Project Folder

Place the project folder anywhere on your computer (e.g. C:\projects\ai_fraud_detection_system or ~/ai_fraud_detection_system). Open your terminal/PowerShell inside this directory:

```
cd path/to/ai_fraud_detection_system
```

Step 3.2 — Create & Activate Virtual Environment

It is strongly recommended to isolate dependencies inside a dedicated Python virtual environment:

```
# Windows (PowerShell or Command Prompt):
python -m venv venv
.\venv\Scripts\activate

# macOS / Linux:
python3 -m venv venv
source venv/bin/activate
```

Step 3.3 — Install Dependencies

Install the required libraries listed in requirements.txt:

```
pip install -r requirements.txt
```

Step 3.4 — Verify / Train Models (Optional & Automated)

The project comes with pre-trained models. However, you can run the automated verification test suite or retrain the models with one command:

```
# Run Automated Unit Verification (100% Passing Check):
python test_system.py

# (Optional) Retrain all models on a fresh 15,000 transaction dataset:
python train_all.py
```

Step 3.5 — Start the Flask Web Application

Launch the backend web server and API engine:

```
python app.py
```

The terminal will indicate that the server is running on **http://127.0.0.1:5000**. Open this URL in your web browser to access the interactive dashboard.

Step 3.6 — Launch Real-Time Payment Stream Simulator

Open a **second terminal window**, activate the virtual environment, and run the real-time payment streaming simulator:

```
cd path/to/ai_fraud_detection_system
# Activate venv (e.g. .\venv\Scripts\activate)
python live_checkout_stream.py
```

This script continuously emits simulated UPI, Card, and Netbanking payments every 2.5 seconds. Switch back to your browser at http://127.0.0.1:5000 to observe real-time charts updating automatically!

4. Web Application User Guide & Modules

The web interface contains five specialized views accessible via the top navigation bar:

Navigation Tab	Functionality & Evaluation Role
Dashboard (/)	Displays total evaluated transactions, approval rates, 2FA challenges, blocked attacks, decision doughnut chart, and fraud categories bar chart.
Live Simulator (/simulator)	Allows entering custom transaction attributes (Amount ■, Category, Device Risk, Velocity, Hour). Features 4 quick preset buttons (UPI Kirana, Card Fraud, ATO, Velocity Burst) to test instant risk evaluation.
Analyst Queue (/queue)	Interactive fraud analyst workstation. Allows manual inspection of flagged transactions with 1-click 'Approve' or 'Block Fraud' override actions.
Model Metrics (/metrics)	Visualizes model performance metrics (Precision, Recall, F1-Score, AUC-PR, ROC-AUC) along with interactive Precision-Recall and ROC curves.
Framework Mapping (/framework)	Detailed enterprise security audit matrix mapping all 5 fraud types to OWASP API Security Top 10 (2023) and PCI DSS v4.0 regulatory controls.

5. Step-by-Step Guide: Uploading Code to Your GitHub Repository

Follow these steps to upload the project code to your personal GitHub account:

Step 5.1 — Create an Empty Repository on GitHub

1. Log in to your GitHub account at **https://github.com**.
2. In the top-right corner, click the **+** icon and select **New repository** (or visit **https://github.com/new**).
3. Enter a **Repository name**, for example: ai-fraud-detection-system.
4. Select **Public** or **Private**.
5. **CRITICAL:** Leave 'Add a README file', '.gitignore', and 'Choose a license' **UNCHECKED** (the project already contains these files).
6. Click the green **Create repository** button.

Step 5.2 — Initialize and Push Code From Terminal

Open a terminal in the project directory and run the following commands sequentially. Replace <YOUR_GITHUB_USERNAME> and <YOUR_REPOSITORY_NAME> with your actual GitHub details:

```
# 1. Initialize git and configure your identity:
git init
git config user.name "Your Name"
git config user.email "your_email@example.com"

# 2. Stage all project files and commit:
git add .
git commit -m "Initial commit of AI-Powered Fraud Detection System"

# 3. Set primary branch to main:
git branch -M main

# 4. Link your personal GitHub repository:
git remote add origin https://github.com/<YOUR_GITHUB_USERNAME>/<YOUR_REPOSITORY_NAME>.git

# 5. Push files to GitHub:
git push -u origin main
```

GIT REMOTE TIP: *If you are using an existing Git repository, you can reset the remote using: git remote set-url origin https://github.com/<YOUR_GITHUB_USERNAME>/<YOUR_REPOSITORY_NAME>.git followed by git push -u origin main.*

6. Step-by-Step Guide: Launching & Hosting on Vercel

The project is pre-configured with vercel.json and an api/index.py serverless WSGI wrapper, making deployment to Vercel completely automated.

Step 6.1 — Log in to Vercel

1. Open your browser and navigate to **https://vercel.com**.
2. Click **Log In** or **Sign Up**.
3. Choose **Continue with GitHub**. This grants Vercel direct access to import your repository.

Step 6.2 — Import the Project Repository

1. On the Vercel Dashboard, click the **Add New...** button in the top-right corner and select **Project**.
2. Under the **Import Git Repository** section, locate your repository (e.g. ai-fraud-detection-system).
3. Click the **Import** button next to your repository name.

Step 6.3 — Configure & Deploy

1. **Project Name:** Keep default or customize (e.g. ai-fraud-detection-system).
2. **Framework Preset:** Leave as **Other** (Vercel automatically detects the Python runtime).
3. **Root Directory:** Leave as ./.
4. Click the blue **Deploy** button.

Step 6.4 — Access Your Live Web Application

Vercel will build the Python environment in approximately 30–60 seconds. Once complete, you will see a **Congratulations!** screen with a preview screenshot and your live public URL (e.g. https://ai-fraud-detection-system.vercel.app). Anyone in the world can now interact with your fraud detection platform securely via HTTPS!

> ***SERVERLESS ARCHITECTURE NOTE:*** *Vercel operates as a stateless Serverless Function environment. While the web dashboard, API evaluation, simulator, and metrics run seamlessly on Vercel, background continuous looping scripts (such as live_checkout_stream.py) should be executed on your local machine targeting either your local host (http://127.0.0.1:5000/api/evaluate) or your live Vercel endpoint.*

7. Frequently Encountered Questions & Troubleshooting

Issue / Question	Resolution & Explanation
Port 5000 Already in Use	If another app is using port 5000, start Flask on a different port: flask run --port 5001 or edit line 185 of app.py to port=5001.
Script Execution Policy Error (Windows)	If PowerShell blocks virtual environment activation, run: Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned then re-activate.
Git Authentication Error on Push	GitHub requires a **Personal Access Token (PAT)** or GitHub Credential Manager instead of your account password. Generate a token under *GitHub Settings > Developer Settings > Personal Access Tokens*.
Modifying Thresholds / Weights	All global decision boundaries (35 / 70) and ensemble weights (45% supervised, 40% sequence, 15% rules) are centralized in config.py for rapid tuning.

Document Information: *Generated for academic distribution, project assessment, and technical review. Contains complete instructions for autonomous replication, source control, and cloud serverless hosting.*